

outer optimization loop

`jax.value_and_grad` / `optax` over $\log \nu$
plus PINN parameters where applicable

drives the gradient-based methods

Tesseract component boundary — component-specific typed `apply` contracts
+ `vector_jacobian_product` where gradients are required

↓ `apply`

↑ VJP (gradient)

burgers_solver

differentiable spectral
Burgers solver (JAX)

VJP = discrete solver adjoint

PINN

`pinn_jax` $\rightleftharpoons$ `pinn_pytorch`

VJP = native autograd
(`jax.grad` / `torch`)

fmpe_posterior

amortized flow posterior

trained offline, apply-only
→ posterior ($\nu, A_{IC}, \varphi_{IC}$)

*Each versioned container defines its own typed contract and follows the same Tesseract `apply`/VJP pattern.
Only the two PINN backends share an identical schema: swapping them is a single `Tesseract.from_image(...)` call.*